PROJECT REPORT

# SmartCart

*A Full Stack E-Commerce Web Application*

Spring Boot 3 • React 19 • MySQL 8

**Submitted for:** Full Stack Java Development Certification

**Training Institute:** SPRK Technologies, Navi Mumbai

**Submitted by:** Darshan Takarkhede

**GitHub Repository:** github.com/NeoAnthem/SmartCart

**Live Application:** smart-cart-opal-sigma.vercel.app

**Date of Submission:** August 2026

# Declaration

I, Darshan Takarkhede, hereby declare that the project titled “SmartCart – Full Stack E-Commerce Web Application” submitted as part of the Full Stack Java Development program at SPRK Technologies is an original record of work carried out by me. The application has been independently designed, developed, tested, and deployed by me using Spring Boot, React, and MySQL, and follows the project scenario and evaluation guidelines issued as part of the course curriculum.

All third-party libraries, frameworks, and cloud services used in this project are duly acknowledged in the Technology Stack and References sections of this report. To the best of my knowledge, this report and the accompanying source code do not infringe upon the intellectual property of any other party.

Place: Mumbai, India
Date: August 2026

**Darshan Takarkhede**

# Acknowledgement

I would like to express my sincere gratitude to the trainers and mentors at SPRK Technologies for their guidance, structured curriculum, and continuous support throughout the Full Stack Java Development program. Their instruction in Core Java, Spring Boot, React, and database design provided the foundation necessary to conceive, build, and deploy SmartCart as a complete, production-grade application.

I am also thankful to the open-source community behind the frameworks and libraries used in this project — Spring Boot, React, Hibernate, and the many supporting packages listed in this report — whose documentation and tooling made independent, self-directed development possible. Finally, I acknowledge the peers and reviewers who tested the deployed application and provided feedback that helped refine its features and usability.

# Table of Contents

# 1. Introduction

## 1.1 Background

Electronic commerce has become one of the most widely adopted models for digital business, and building a production-grade online shopping platform requires the integration of several full stack engineering disciplines: secure authentication, relational data modelling, RESTful service design, cloud storage, transactional workflows, and a responsive user interface. SmartCart was undertaken as a capstone project for the Full Stack Java Development program at SPRK Technologies to demonstrate proficiency across each of these disciplines within a single, cohesive, and independently deployed application.

Rather than implement an isolated set of academic exercises, the project was scoped to mirror how a real retail organisation would commission an online storefront — with distinct customer-facing and administrative experiences, a documented REST API, a normalised database, and a live, cloud-hosted deployment that can be evaluated end-to-end.

## 1.2 Problem Statement

A retail company, SmartCart, requires a modern online shopping platform that allows customers to browse products, manage their accounts, add items to a cart, place orders, make payments, and track deliveries — while administrators need the ability to manage products, inventory, customers, and orders. The objective of this project is to design and develop such a system using a layered, REST-based architecture built with Spring Boot, React, and MySQL, following the Controller–Service–Repository pattern, DTO-based data exchange, and centralised exception handling.

## 1.3 Objectives of the Project

- Design and implement a secure, role-based e-commerce backend using Spring Boot 3 and Spring Security.
- Build a responsive, component-driven single-page frontend in React 19 that consumes REST APIs exclusively.
- Model a normalised relational schema in MySQL covering users, products, categories, cart items, orders, order items, payments, reviews, wishlist, and coupons.
- Implement JWT-based authentication and role-based access control (RBAC) for Customer and Admin roles.
- Integrate third-party cloud services — Cloudinary for image storage, Gmail SMTP for transactional email, and iText7 for PDF invoice generation.
- Provide an administrative dashboard with sales analytics, inventory alerts, and performance reports.
- Document the REST API using springdoc-openapi (Swagger UI) for verifiability and third-party consumption.
- Deploy the completed application to production cloud infrastructure (Vercel, Render, and Railway) rather than a local-only demo.

## 1.4 Scope of the Project

The current scope covers the full customer purchase journey (registration through checkout, invoicing, and order tracking) and a complete administrative back-office (product, category, inventory, order, and user management, plus analytics). Payment processing is implemented as a simulated gateway that models real-world success and failure outcomes for demonstration purposes; integration with a live payment processor such as Razorpay or Stripe, along with containerisation, automated CI/CD, and automated test suites, are identified as future scope in Chapter 15 rather than included in the current submission.

# 2. Requirement Analysis

## 2.1 Functional Requirements

### Customer-Facing Requirements

- User registration and login with encrypted password storage.
- Browse, search, filter (by category), and sort (by price) the product catalogue, including a paginated product listing.
- View detailed product information, including images, description, price, stock, and customer reviews, through a details modal.
- Maintain a personal shopping cart with add, update-quantity, and remove operations.
- Maintain a wishlist of saved products independent of the cart.
- Apply discount coupon codes at checkout and view the resulting price breakdown.
- Place an order (checkout), view order history, track order status, and cancel an eligible order.
- Download a system-generated PDF invoice for a placed order.
- Submit a rating and written review for a purchased product.
- View and update profile details, change password, and view personal order/wishlist statistics.

### Administrative Requirements

- Secure, role-restricted dashboard summarising users, products, orders, revenue, and order-status breakdown.
- Full CRUD management of products, including image upload and stock-level updates.
- Full CRUD management of product categories.
- Order oversight: view all customer orders and update order status (e.g., pending → shipped → delivered).
- User management: view registered users, promote/change roles, and remove accounts.
- Low-stock alerts for inventory that has fallen below a safe threshold.
- Sales and performance reporting: total revenue, average order value, monthly revenue trend, and per-product performance.

## 2.2 Non-Functional Requirements

- Security: all state-changing and user-specific endpoints must be authenticated via JWT and authorised by role.
- Performance: product listings should support pagination and lazy-loaded routes to keep initial load times low.
- Scalability: a layered architecture (Controller / Service / Repository) that allows independent scaling and testing of each concern.
- Availability: the application should be reachable as a live, cloud-hosted deployment rather than a local-only build.
- Usability: a consistent, responsive interface that adapts across desktop, tablet, and mobile viewports.
- Maintainability: consistent naming conventions, DTO-based contracts between layers, and centralised exception handling.

## 2.3 Hardware & Software Requirements

| Category | Requirement |
|---|---|
| Development OS | Windows / Linux / macOS (any OS capable of running a JDK and Node.js) |
| Backend Runtime | Java Development Kit (JDK) 21 |
| Backend Framework | Spring Boot 3.5.x (Maven build) |
| Frontend Runtime | Node.js (18+) with npm, Vite build tooling |
| Database | MySQL 8 |
| IDE / Tools | IntelliJ IDEA / VS Code, Postman, MySQL Workbench, Git |
| Minimum RAM (dev machine) | 8 GB (16 GB recommended for running backend, frontend, and MySQL together) |
| Browser | Any modern evergreen browser (Chrome, Edge, Firefox) for the deployed React application |
| Production Hosting | Vercel (frontend), Render (backend), Railway (MySQL) — no local infrastructure required to evaluate the live system |

# 3. Technology Stack

SmartCart was intentionally built with a modern, production-representative stack rather than the minimum needed to satisfy the assignment brief, so that the finished project also functions as a portfolio-grade demonstration of current full stack practice.

| Layer | Technology | Purpose |
|---|---|---|
| Backend Language | Java 21 | Core backend programming language |
| Backend Framework | Spring Boot 3.5.15 | Application framework, auto-configuration, embedded server |
| Security | Spring Security + JJWT 0.11.5 | Authentication, authorization, JWT issuing/parsing |
| Persistence | Spring Data JPA + Hibernate | ORM and repository abstraction over MySQL |
| Password Hashing | BCrypt (Spring Security Crypto) | One-way password encryption |
| Validation | Spring Boot Starter Validation (Jakarta Bean Validation) | DTO-level field validation (@NotBlank, @Email, @Size, etc.) |
| Email | Spring Boot Starter Mail (Gmail SMTP) | Order-confirmation email notifications |
| Image Storage | Cloudinary SDK 1.39.0 | Cloud image hosting with on-the-fly optimisation (f_auto, q_auto) |
| PDF Generation | iText7 (itext7-core 7.2.5) | Dynamic PDF invoice generation for completed orders |
| API Documentation | springdoc-openapi 2.8.8 (Swagger UI) | Interactive, auto-generated REST API documentation |
| Database | MySQL 8 | Primary relational data store (10 tables) |
| Build Tool (Backend) | Maven | Dependency management and build lifecycle |
| Boilerplate Reduction | Lombok | Reduces boilerplate getters/setters/constructors |
| Frontend Library | React 19 | Component-based single-page application UI |
| Build Tool (Frontend) | Vite | Fast dev server and production bundler |
| Routing | React Router DOM 7 | Client-side routing and protected routes |
| HTTP Client | Axios | REST API integration from the frontend |
| Data Visualisation | Recharts | Revenue and analytics charts on the admin dashboard |
| UI Feedback | React Toastify, SweetAlert2 | Toast notifications and confirmation dialogs |
| Iconography | React Icons, Lucide React, FontAwesome Free | Consistent iconography across the UI |
| Styling | CSS3 (Flexbox & Grid) | Custom responsive, glassmorphism-styled layouts |
| Frontend Hosting | Vercel | Production hosting of the React build |
| Backend Hosting | Render | Production hosting of the Spring Boot API |
| Database Hosting | Railway | Managed MySQL instance for production |

# 4. System Architecture

## 4.1 Architectural Overview

SmartCart follows a classic three-tier, layered architecture on the backend (Controller → Service → Repository) fronted by a completely decoupled React single-page application. All communication between the frontend and backend takes place exclusively over versioned, JSON-based REST endpoints secured by a stateless JWT filter chain. Cloud services for image storage, email, PDF generation, and API documentation are integrated at the service layer, keeping controllers thin and focused purely on request/response handling.

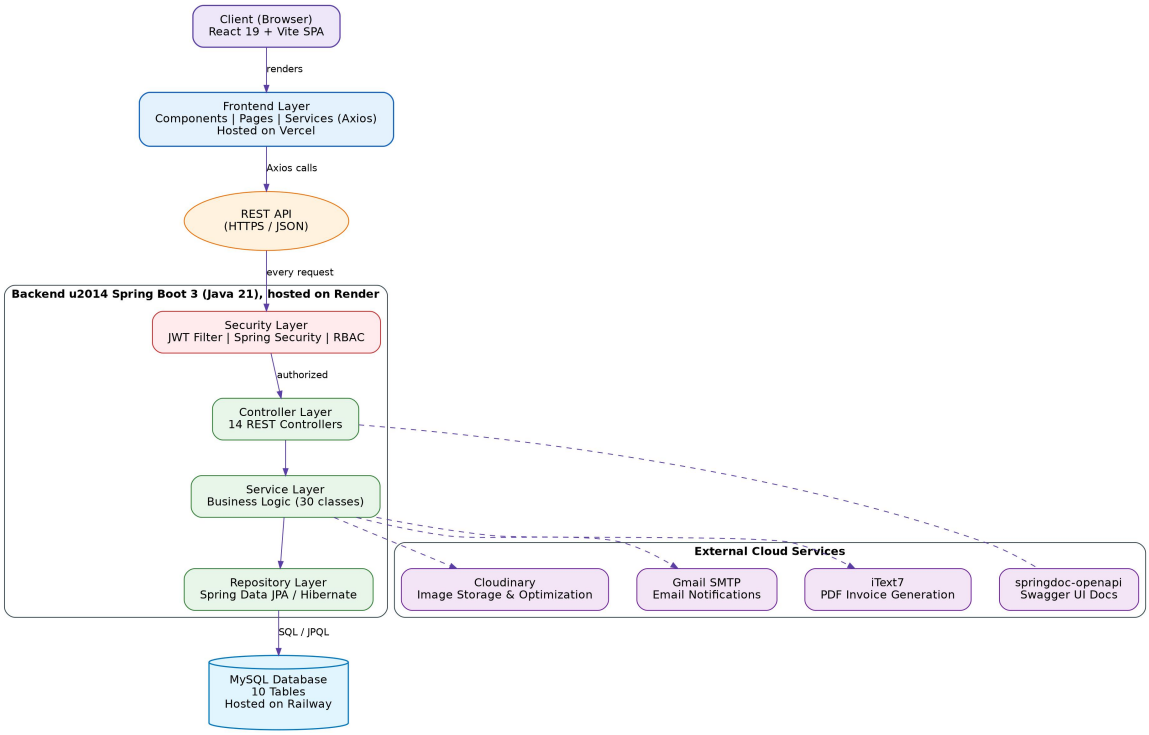


*Figure 4.1 — SmartCart layered system architecture*

## 4.2 Layered Architecture Explained

### Frontend Layer (React 19 + Vite)

Organised into components/, pages/, services/, styles/, and utils/ directories. Reusable UI primitives (glass-panel cards, loaders, confirmation dialogs, pagination) live in components/, route-level screens live in pages/, and all Axios calls are centralised in services/ so that API base URLs and JWT headers are configured in exactly one place. Route-based code splitting and React lazy-loading are used to keep the initial bundle small.

### Controller Layer

Fifteen REST controllers accept HTTP requests, delegate immediately to the service layer, and return DTOs or entities as JSON. Role restrictions are declared directly on controller methods using Spring Security's @PreAuthorize annotation (for example, @PreAuthorize("hasRole('ADMIN')") on product-management and reporting endpoints), keeping authorization rules visible and close to the endpoint they protect.

### Service Layer

Sixteen service interfaces (with matching implementation classes) contain all business logic: cart-to-order conversion, coupon discount calculation, stock validation, dashboard aggregation, invoice generation, and email dispatch. Checkout is wrapped in a single @Transactional boundary so that order creation, stock adjustment, and coupon application either fully succeed or fully roll back together.

### Repository Layer

Ten Spring Data JPA repositories provide the persistence contract for each entity, relying on Hibernate-generated queries and a small number of custom derived-query methods (for example, locating a user by email, or filtering products by category and price).

### Security Layer

A custom JwtAuthenticationFilter is registered ahead of Spring Security's UsernamePasswordAuthenticationFilter, validating the bearer token on every request, loading the authenticated user via a CustomUserDetailsService, and populating the security context so that @PreAuthorize checks and method-level RBAC work consistently across all fifteen controllers.

### External Integrations

The service layer talks to four external systems: Cloudinary for image upload/optimisation, Gmail SMTP for order-confirmation email, iText7 for generating downloadable PDF invoices, and springdoc-openapi for exposing a live Swagger UI at /swagger-ui/index.html.

# 5. Database Design

## 5.1 Entity-Relationship Diagram

The schema is fully normalised and consists of ten tables hosted on a managed MySQL 8 instance on Railway. Product catalogue data (categories, products) is separated from transactional data (orders, order_items, payments), and each user-specific relationship — cart, wishlist, and reviews — is modelled as its own join-style table referencing both users and products.

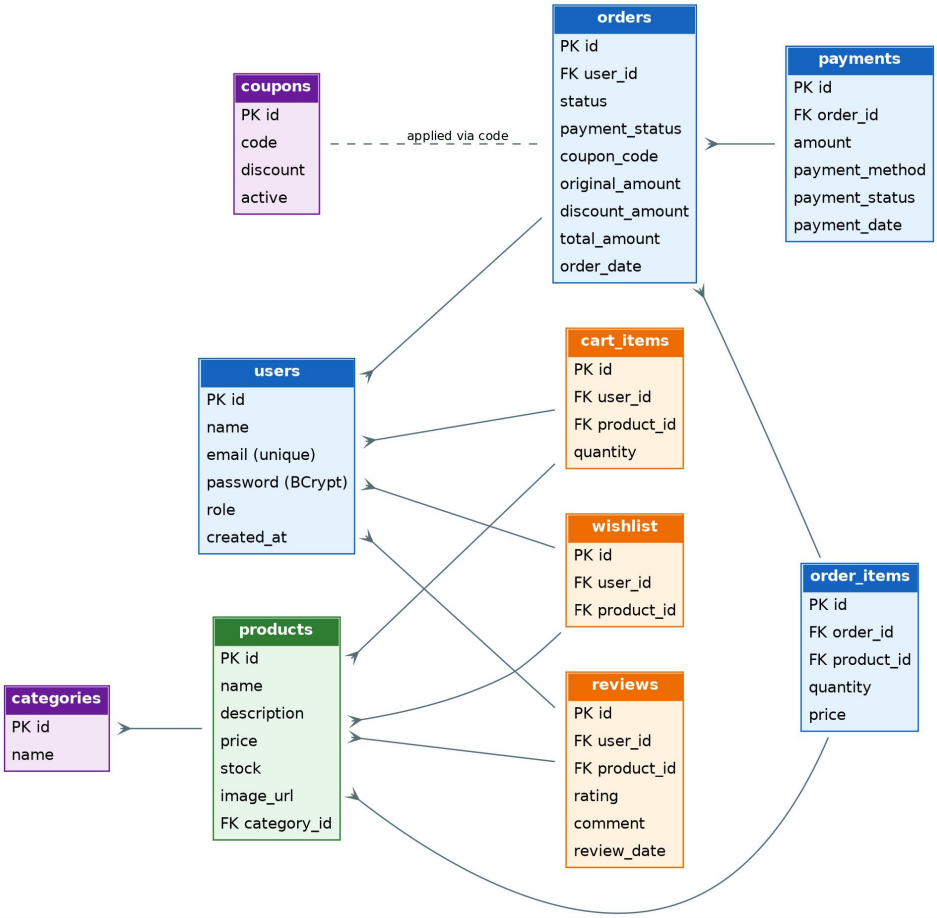


*Figure 5.1 — SmartCart entity-relationship diagram*

## 5.2 Table Descriptions

| Table | Purpose | Key Relationships |
|---|---|---|
| users | Stores registered customers and administrators, including BCrypt-hashed passwords and role. | Referenced by cart_items, wishlist, reviews, and orders |
| categories | Product category taxonomy (e.g., Electronics, Apparel). | Referenced by products |
| products | Product catalogue: name, description, price, stock, image URL, category. | Belongs to categories; referenced by cart_items, wishlist, reviews, order_items |
| cart_items | A user's active shopping cart lines prior to checkout. | Links users and products (many-to-one each) |
| wishlist | Products a user has saved for later. | Links users and products |
| reviews | Star rating and written comment left by a user for a product. | Links users and products |
| coupons | Discount codes with a percentage discount and an active flag. | Applied to orders via coupon_code at checkout |
| orders | A placed order: status, payment status, coupon applied, original/discounted/total amount, order date. | Belongs to a user; has many order_items and payments |
| order_items | Line items for an order, capturing product, quantity, and price at time of purchase. | Belongs to an order and a product |
| payments | Payment attempt against an order: amount, method, status, timestamp. | Belongs to an order |

All primary keys are auto-incrementing surrogate identifiers, and foreign keys enforce referential integrity between users, products, and the transactional tables. Storing price and quantity directly on order_items (rather than only referencing the live product price) preserves an accurate historical record of what was actually charged, even if a product's price changes later.

# 6. Module-Wise Feature Description

## 6.1 Customer Modules

### Authentication

Customers register with name, email, and password (validated with @NotBlank, @Email, and a minimum length of six characters). Passwords are hashed with BCrypt before storage. Login returns a signed JWT that the frontend attaches to every subsequent request.

### Product Browsing, Search & Filtering

The Products page supports free-text search, category filtering, price-ascending/descending sorting, and a paginated view (GET /api/products/paged) for larger catalogues. Selecting a product opens a details modal showing images, description, stock, and existing reviews.

### Cart & Wishlist

Cart items and wishlist entries are both modelled per-user. Customers can add a product to either list, adjust cart quantities, or remove an item, independent of one another — a product can sit in a wishlist without being added to the cart.

### Coupons & Checkout

At checkout, an optional coupon code is validated server-side (GET /api/coupons/validate/{code}); if active, the discount percentage is applied to the cart subtotal before the order is created. The entire checkout — cart validation, discount calculation, order and order-item creation, and confirmation email — executes inside a single transactional service method.

### Orders, Invoices & Cancellation

Customers can view their order history, cancel an order that is still eligible for cancellation, and download a PDF invoice (generated on demand with iText7) for any placed order via GET /api/orders/invoice/{orderId}.

### Reviews

Customers can submit a star rating and comment for a product (POST /api/reviews), and all reviews for a product are visible to every visitor (GET /api/reviews/{productId}).

### Profile Management

Customers can update their profile details, change their password, and view summary statistics — total orders placed, wishlist item count, cart item count, and member-since date — on their profile page.

## 6.2 Admin Modules

### Dashboard

A single aggregated endpoint (GET /api/dashboard) returns total users, total products, total orders, total revenue, a breakdown of orders by status (pending / shipped / delivered / cancelled), monthly and daily revenue series for charting, and the most recent orders — all rendered on one screen using Recharts.

### Product & Category Management

Admins have full CRUD control over products and categories, including direct image upload to Cloudinary (POST /api/products/upload) and dedicated stock-update (PUT /api/products/{id}/stock) and low-stock query (GET /api/products/low-stock) endpoints that power the dashboard's restock alerts.

### Order Management

Admins can view every order placed on the platform (GET /api/orders/admin) and progress its status (PUT /api/orders/{id}/status), independent of the customer-facing order list.

### User Management

Admins can list all registered users, change a user's role, or remove an account entirely, through a dedicated, role-locked controller (/api/admin/users/**).

### Reports & Analytics

A separate reporting module exposes total revenue and average/highest order value (GET /api/reports/sales), a month-by-month revenue trend (GET /api/reports/monthly-revenue), and per-product performance — units sold, revenue, average price, and remaining stock (GET /api/reports/product-performance) — to support merchandising decisions.

# 7. REST API Documentation

The backend exposes 49 REST endpoints across 15 controllers, all documented interactively via Swagger UI at /swagger-ui/index.html on the deployed backend. The tables below summarise every endpoint grouped by functional area. “Public” endpoints require no token; “Customer” and “Admin” endpoints require a valid JWT bearer token with the corresponding role.

## 7.1 Authentication — /api/auth

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /api/auth/register | Public | Register a new customer account |
| POST | /api/auth/login | Public | Authenticate and receive a signed JWT |

## 7.2 Products & Categories — /api/products, /api/categories

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /api/products | Admin | Create a new product |
| GET | /api/products | Public | List all products |
| GET | /api/products/{id} | Public | Get a single product by ID |
| PUT | /api/products/{id} | Admin | Update a product |
| DELETE | /api/products/{id} | Admin | Delete a product |
| GET | /api/products/category/{categoryId} | Public | List products in a category |
| GET | /api/products/search | Public | Keyword search across products |
| GET | /api/products/sort/price-asc | Public | List products sorted by price ascending |
| GET | /api/products/sort/price-desc | Public | List products sorted by price descending |
| GET | /api/products/paged | Public | Paginated product listing |
| PUT | /api/products/{id}/stock | Admin | Update stock quantity for a product |
| GET | /api/products/low-stock | Admin | List products below the stock threshold |
| POST | /api/products/upload | Admin | Upload a product image to Cloudinary |
| POST | /api/categories | Admin | Create a category |
| GET | /api/categories | Public | List all categories |
| PUT | /api/categories/{id} | Admin | Update a category |
| DELETE | /api/categories/{id} | Admin | Delete a category |

## 7.3 Cart, Wishlist & Reviews

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /api/cart | Customer | Add a product to the cart |
| GET | /api/cart | Customer | View the current user's cart |
| DELETE | /api/cart/{cartId} | Customer | Remove an item from the cart |
| PUT | /api/cart/{cartId} | Customer | Update quantity of a cart item |

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /api/wishlist | Customer | Add a product to the wishlist |
| GET | /api/wishlist | Customer | View the current user's wishlist |
| DELETE | /api/wishlist/{id} | Customer | Remove an item from the wishlist |
| POST | /api/reviews | Customer | Submit a product review and rating |
| GET | /api/reviews/{productId} | Public | List reviews for a product |

### 7.4 Orders, Payments & Coupons

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /api/orders/checkout | Customer | Convert the current cart into an order (with optional coupon) |
| GET | /api/orders | Customer | View the current user's order history |
| GET | /api/orders/admin | Admin | View every order placed on the platform |
| PUT | /api/orders/{id}/status | Admin | Update the status of an order |
| PUT | /api/orders/cancel/{id} | Customer | Cancel an eligible order |
| GET | /api/orders/invoice/{orderId} | Customer | Download the PDF invoice for an order |
| POST | /api/payments | Customer | Process payment for an order |
| GET | /api/payments | Customer | View payment records |
| POST | /api/coupons | Admin | Create a discount coupon |
| GET | /api/coupons/validate/{code} | Public | Validate a coupon code and return its discount |

### 7.5 Profile, Dashboard, Reports & User Management

| Method | Endpoint | Access | Description |
|---|---|---|---|
| PUT | /api/profile | Customer/Admin | Update profile information |
| PUT | /api/profile/password | Customer/Admin | Change account password |
| GET | /api/profile/stats | Customer/Admin | View personal order/wishlist/cart statistics |
| GET | /api/dashboard | Admin | Aggregated admin dashboard metrics |
| GET | /api/reports/sales | Admin | Total revenue, average and highest order value |
| GET | /api/reports/monthly-revenue | Admin | Monthly revenue trend series |
| GET | /api/reports/product-performance | Admin | Units sold, revenue and stock per product |
| GET | /api/admin/users | Admin | List all registered users |
| DELETE | /api/admin/users/{id} | Admin | Delete a user account |
| PUT | /api/admin/users/{id}/role | Admin | Change a user's role |
| GET | /api/test | Public | Health-check endpoint |

# 8. Security Implementation

Security is enforced consistently across all fifteen controllers rather than left to ad-hoc checks inside individual handlers, using a combination of stateless JWT authentication, method-level role authorisation, and encrypted credential storage.

## 8.1 JWT Authentication Flow

- A customer or admin authenticates via POST /api/auth/login with an email and password.
- On success, the backend issues a signed JWT (via the JJWT library) that encodes the user's identity and is returned to the client.
- The React frontend stores the token and attaches it as an Authorization: Bearer <token> header on every subsequent Axios request.
- A custom JwtAuthenticationFilter intercepts every incoming request ahead of Spring Security's standard authentication filter, validates the token's signature and expiry, and loads the corresponding user via CustomUserDetailsService.
- Session creation policy is set to STATELESS — the server holds no session state, so the architecture scales horizontally without sticky sessions.

## 8.2 Role-Based Access Control (RBAC)

Two roles are defined — ROLE_ADMIN and ROLE_CUSTOMER. Method-level authorisation is enabled via @EnableMethodSecurity, and individual controller methods are annotated with @PreAuthorize("hasRole('ADMIN')") or @PreAuthorize("hasAnyRole('CUSTOMER','ADMIN')") as appropriate, so authorisation rules are declared directly next to the endpoint they protect rather than centralised in a single hard-to-audit configuration block.

## 8.3 Password Security

All passwords are hashed using BCryptPasswordEncoder before being persisted; plaintext passwords are never written to the database or logged.

## 8.4 CORS Configuration

Cross-Origin Resource Sharing is explicitly restricted to two known origins — the local development frontend (http://localhost:5173) and the production Vercel deployment — rather than a permissive wildcard, with GET, POST, PUT, DELETE, and OPTIONS allowed and credentials enabled.

## 8.5 Protected & Public Routes

The security filter chain explicitly whitelists a small set of public routes — authentication endpoints, Swagger UI and OpenAPI documentation, static images, public read access to products, and public read access to reviews — while every other request must present a valid, authenticated JWT (.anyRequest().authenticated()).

## 8.6 Environment & Secrets Management

Cloudinary credentials, Gmail SMTP credentials, database credentials, and the JWT signing secret are all externalised as environment variables rather than committed to source control, consistent with twelve-factor application practice for cloud-deployed services.

# 9. Validation & Exception Handling

## 9.1 Request Validation

Incoming request DTOs use Jakarta Bean Validation annotations to reject malformed input before it reaches business logic. For example, the registration request enforces a non-blank name, a syntactically valid email, and a minimum six-character password:

- @NotBlank(message = "Name is required")
- @Email(message = "Invalid email")
- @Size(min = 6, message = "Password must be at least 6 characters")

Similar constraints are applied across the other request DTOs (login, payment, profile update, password change), keeping validation declarative and colocated with the field it constrains.

## 9.2 Centralised Exception Handling

A single @RestControllerAdvice class, GlobalExceptionHandler, intercepts exceptions thrown anywhere in the service or controller layers and converts them into consistent JSON error responses with appropriate HTTP status codes, rather than leaking stack traces to the client:

| Exception | HTTP Status | Typical Trigger |
|---|---|---|
| UserAlreadyExistsException | 400 Bad Request | Registration attempted with an email already in use |
| InvalidCredentialsException | 401 Unauthorized | Login attempted with an incorrect password |
| CategoryNotFoundException | 404 Not Found | Operation on a category ID that does not exist |
| ProductNotFoundException | 404 Not Found | Operation on a product ID that does not exist |
| MethodArgumentNotValidException | 400 Bad Request | A @Valid request DTO fails one or more field constraints |
| RuntimeException (generic fallback) | 400 Bad Request | Business-rule violations, e.g., checkout with an empty cart or an invalid coupon |

This gives every API consumer — the React frontend, Swagger UI, or a tool such as Postman — a predictable, structured error contract regardless of which layer of the application actually raised the problem.

# 10. User Interface & Screenshots

The interface uses a consistent dark, glassmorphism-inspired design language across both the customer storefront and the admin console — translucent panels, soft purple-to-cyan gradient accents, and a shared set of reusable components (modals, loaders, pagination, confirmation dialogs) so that new screens remain visually consistent without duplicating styling logic. The layout is fully responsive across desktop, tablet, and mobile breakpoints using CSS Flexbox and Grid.

## 10.1 Customer Interface

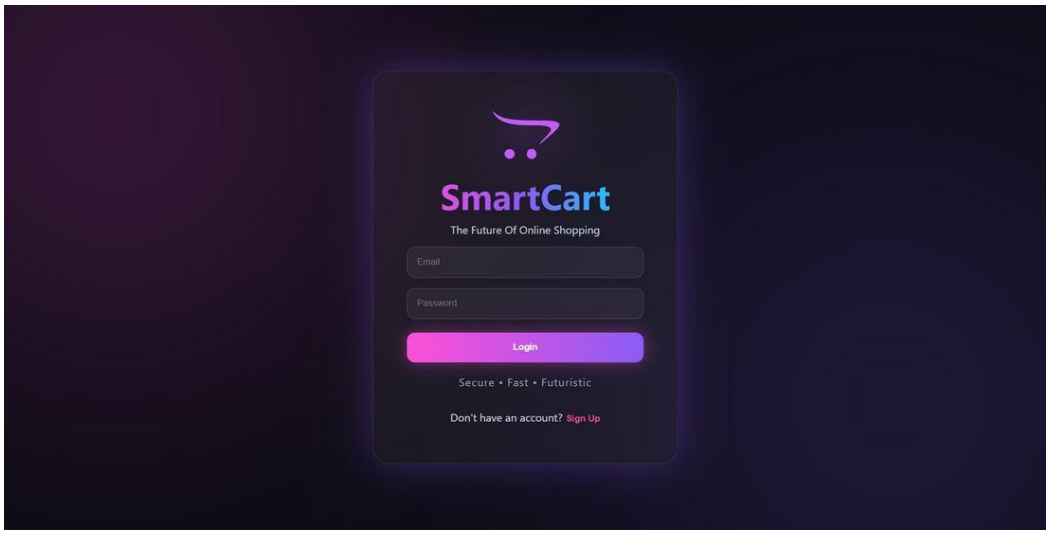


*Figure 10.1 — Login page*

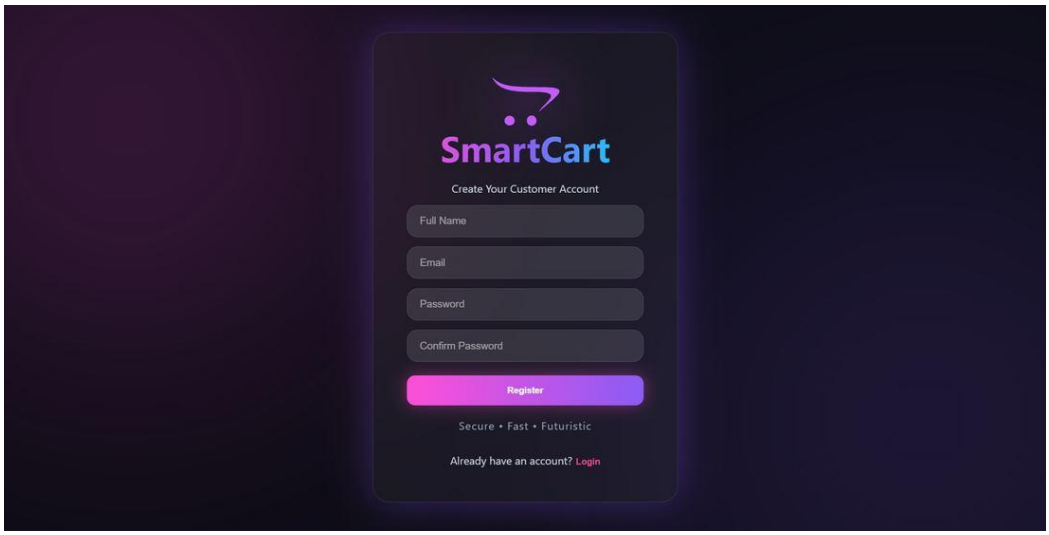


*Figure 10.2 — Registration page*

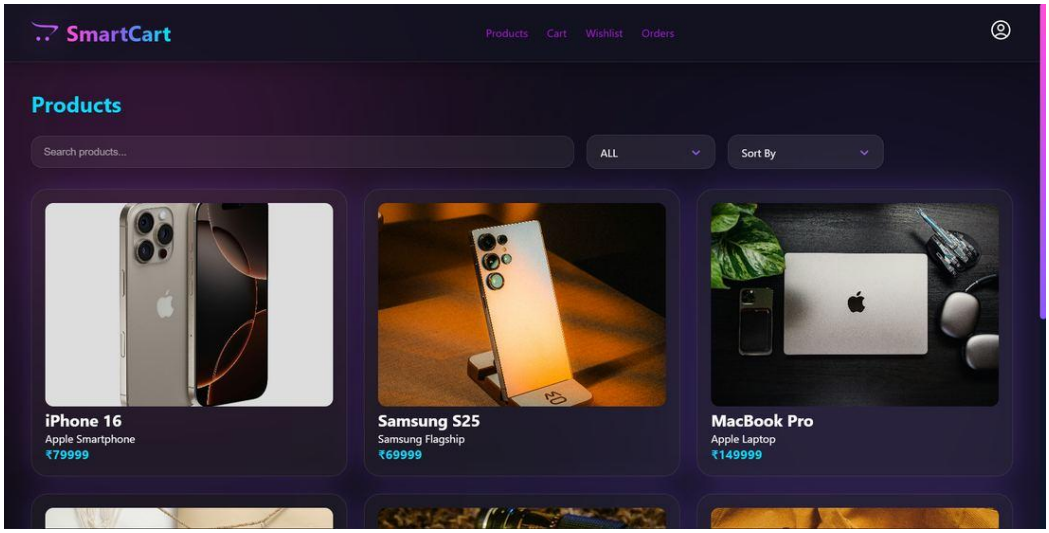


*Figure 10.3 — Product catalogue with search, category filter, and sorting*

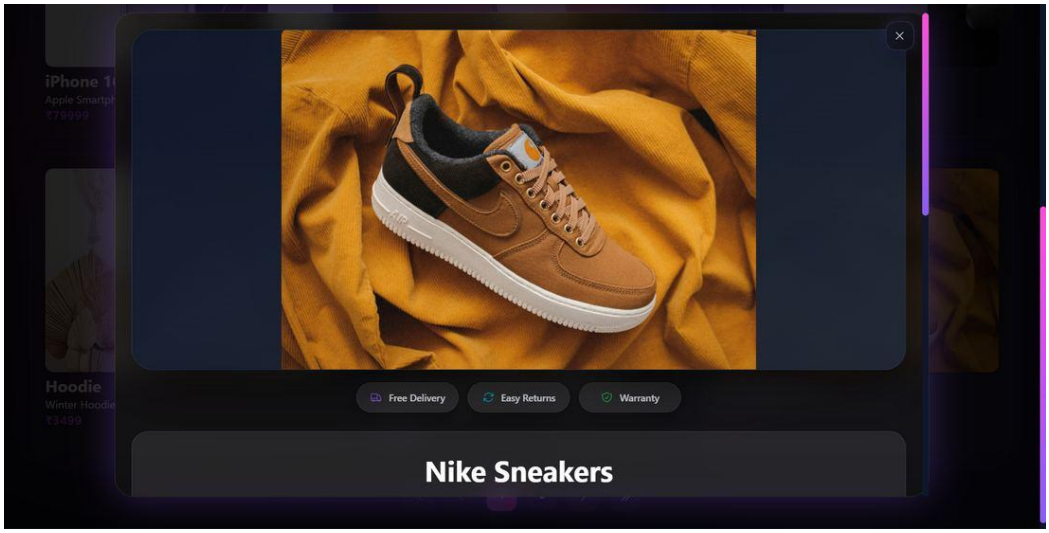


*Figure 10.4 — Product details modal with description and reviews*

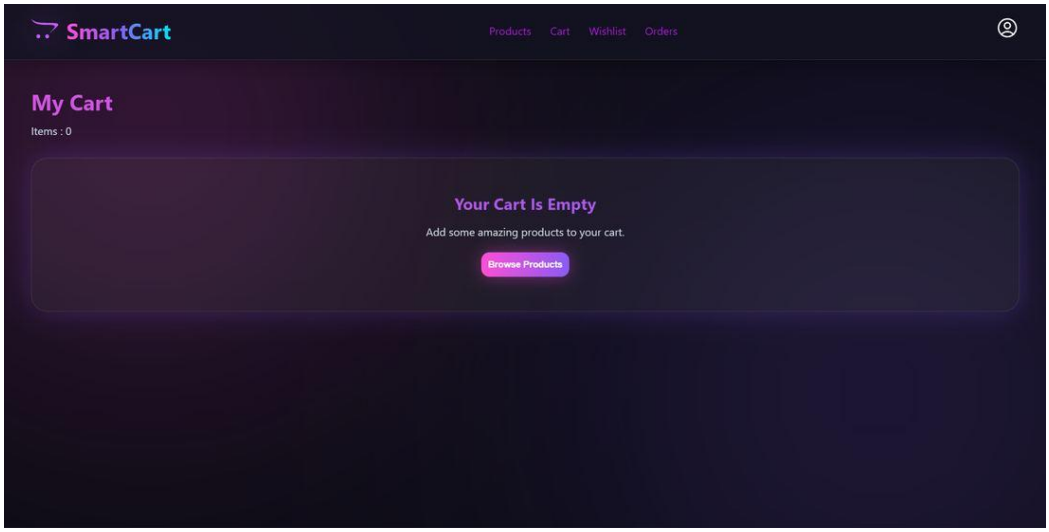


*Figure 10.5 — Shopping cart with quantity controls and order summary*

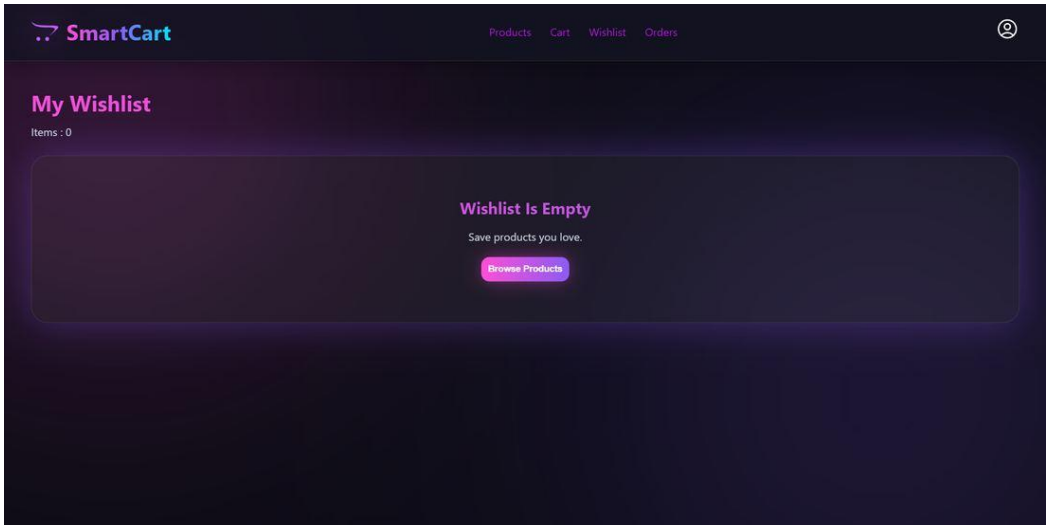


*Figure 10.6 — Wishlist page*

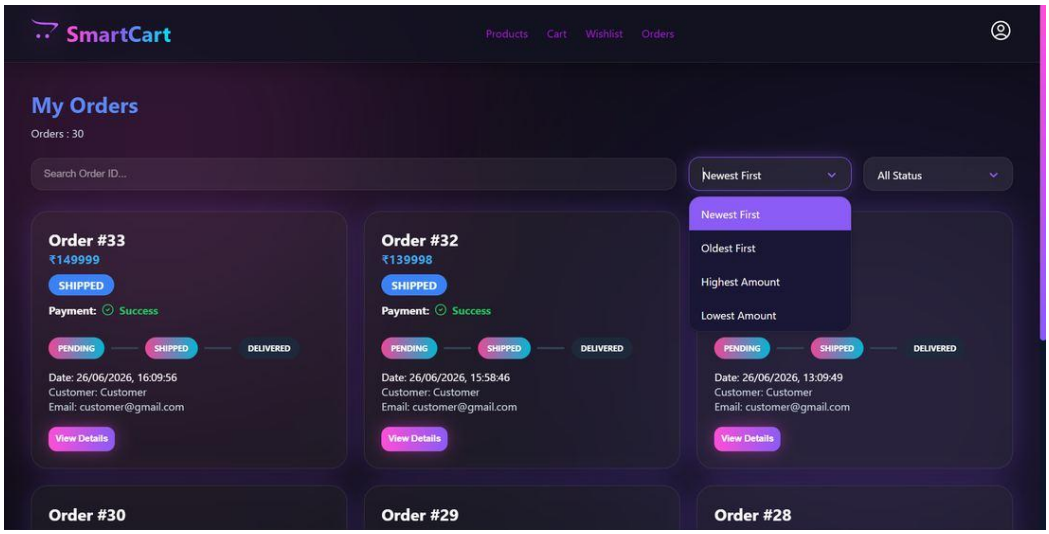


*Figure 10.7 — Customer order history with status tracking*

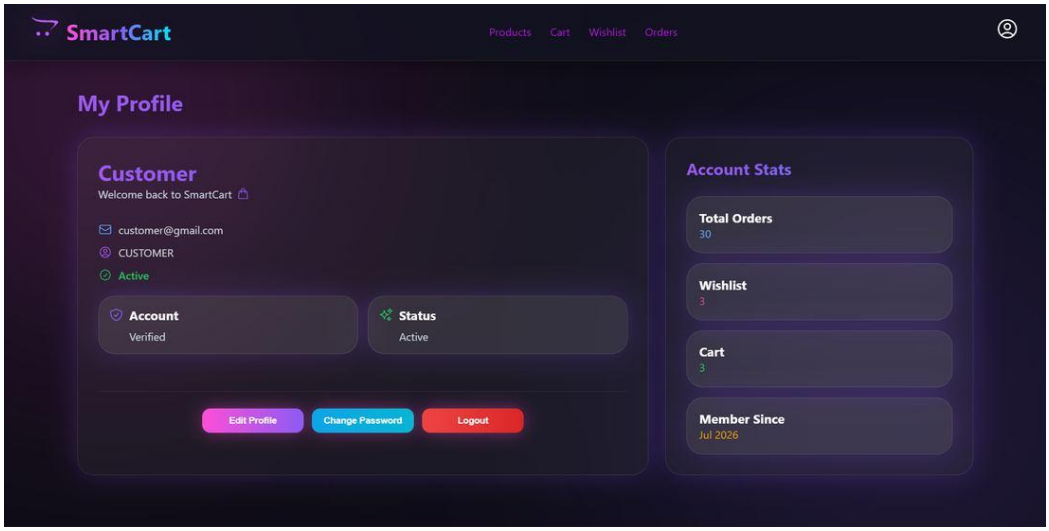


*Figure 10.8 — Customer profile with account statistics*

## 10.2 Admin Interface

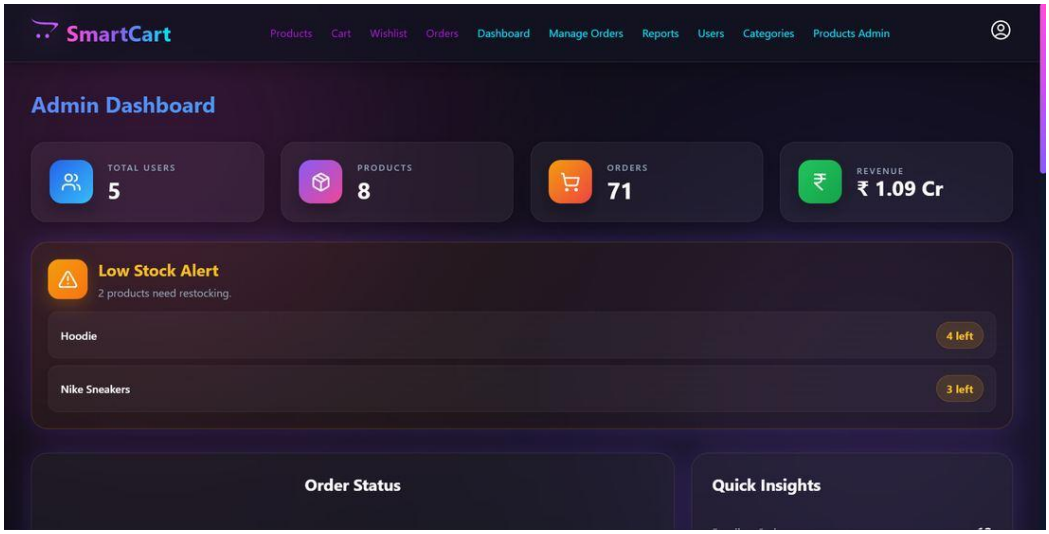


*Figure 10.9 — Admin dashboard: revenue, order status, and low-stock alerts*

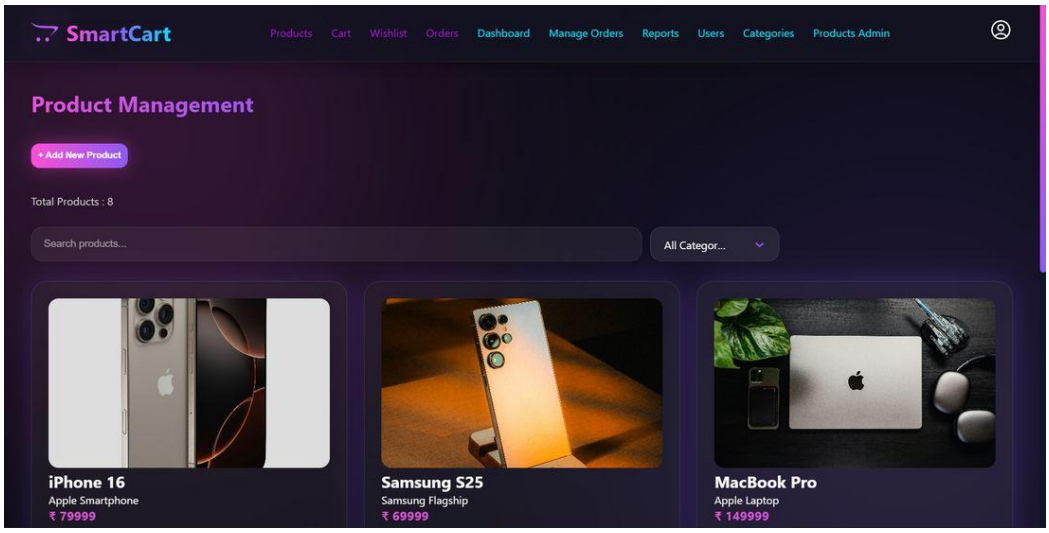


*Figure 10.10 — Admin product management*

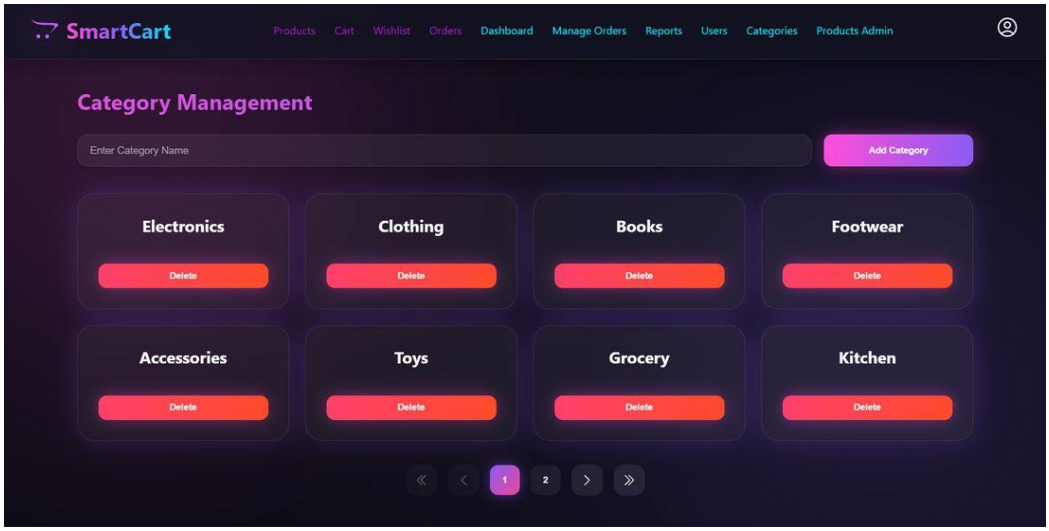


*Figure 10.11 — Admin category management*

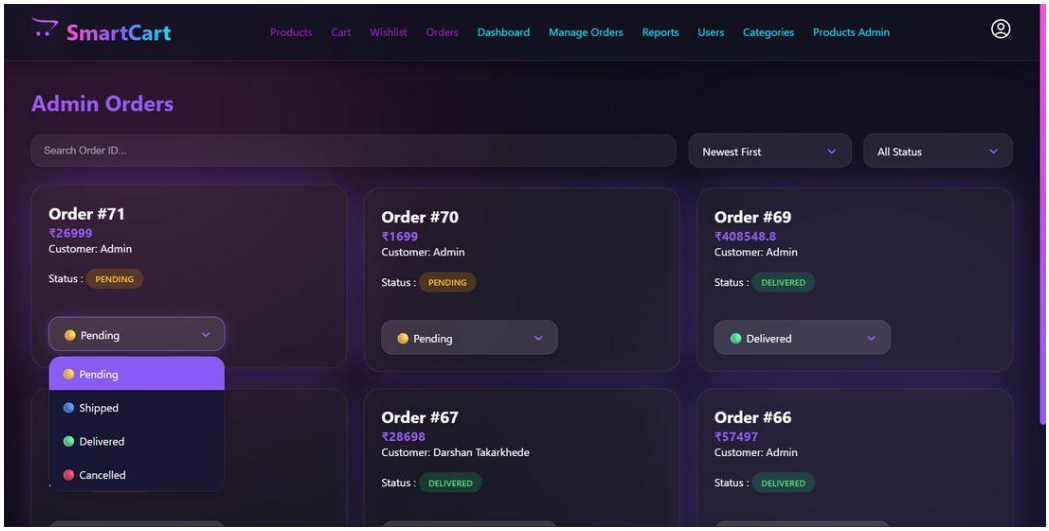


*Figure 10.12 — Admin order management with status updates*

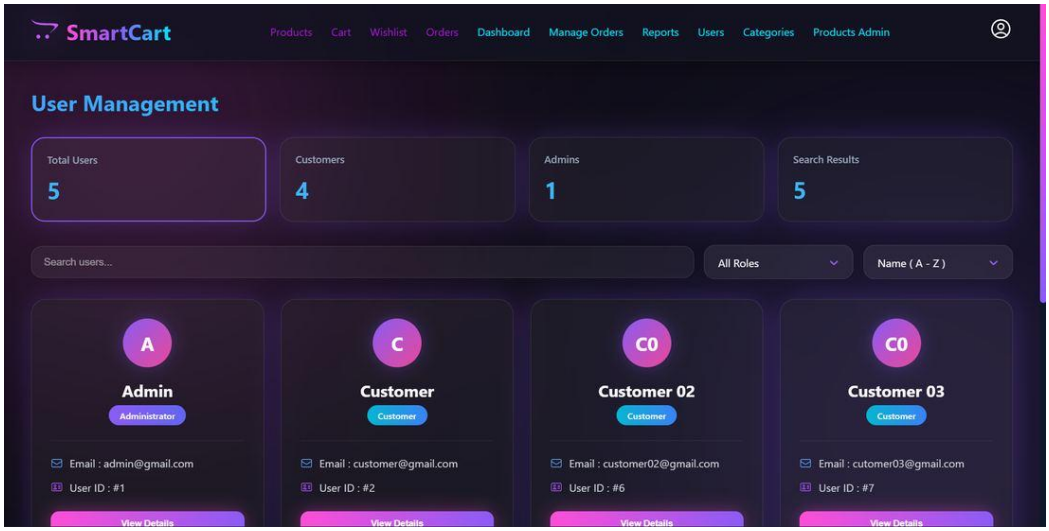


*Figure 10.13 — Admin user management*

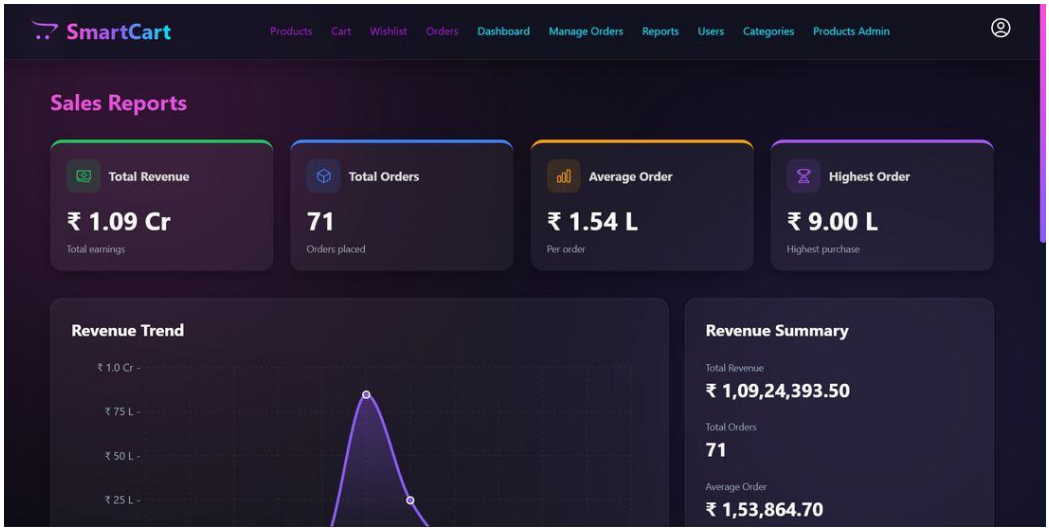


*Figure 10.14 — Sales and product performance reports*

# 11. Testing

## 11.1 Testing Approach

Given the scope of a single-developer capstone project, testing was carried out primarily through structured manual functional testing across both the customer and admin flows, exercised directly against the deployed environment and cross-checked through the Swagger UI and Postman for individual endpoints. The Spring Boot project is scaffolded with spring-boot-starter-test and spring-security-test, and an application-context load test (SmartcartBackendApplicationTests) verifies the Spring context wires correctly; expanding this into a full JUnit/Mockito unit and integration test suite is identified as future scope in Chapter 15.

## 11.2 Test Cases

| ID | Module | Test Scenario | Expected Result | Status |
|---|---|---|---|---|
| TC-01 | Auth | Register with a new, valid email/password | Account created; success response returned | Pass |
| TC-02 | Auth | Register with an email already in use | 400 Bad Request — “User already exists” | Pass |
| TC-03 | Auth | Login with correct credentials | 200 OK with a signed JWT returned | Pass |
| TC-04 | Auth | Login with an incorrect password | 401 Unauthorized — invalid credentials | Pass |
| TC-05 | Security | Call a protected endpoint with no JWT | 401/403 — access denied | Pass |
| TC-06 | Security | Customer calls an Admin-only endpoint | 403 Forbidden — RBAC enforced | Pass |
| TC-07 | Products | Search products by keyword | Matching products returned | Pass |
| TC-08 | Products | Filter products by category | Only products in that category returned | Pass |
| TC-09 | Products | Sort products by price (asc/desc) | Results correctly ordered | Pass |
| TC-10 | Cart | Add a product to the cart | Cart item created with quantity 1 | Pass |
| TC-11 | Cart | Update the quantity of a cart item | Quantity updated; total recalculated | Pass |
| TC-12 | Wishlist | Add and remove a product from the wishlist | Wishlist reflects the change immediately | Pass |
| TC-13 | Checkout | Checkout with a valid, active coupon | Discount applied; order total reduced accordingly | Pass |
| TC-14 | Checkout | Checkout with an expired/inactive coupon | 400 Bad Request — “Coupon Expired” | Pass |
| TC-15 | Checkout | Checkout with an empty cart | 400 Bad Request — “Cart is Empty” | Pass |
| TC-16 | Orders | Cancel a recently placed, eligible order | Order status updated to cancelled | Pass |
| TC-17 | Orders | Download invoice for a completed order | PDF invoice downloaded successfully | Pass |
| TC-18 | Payments | Simulate a payment attempt | Payment recorded as SUCCESS or FAILED; order updated to match | Pass |
| TC-19 | Reviews | Submit a rating and comment for a | Review saved and visible on the product | Pass |

| ID | Module | Test Scenario | Expected Result | Status |
|---|---|---|---|---|
| | | product | page | |
| TC-20 | Admin | Add a new product with an uploaded image | Product created; image served from Cloudinary | Pass |
| TC-21 | Admin | Update stock and trigger a low-stock alert | Dashboard “Low Stock Alert” lists the product | Pass |
| TC-22 | Admin | Update an order's status | New status reflected in customer's order history | Pass |
| TC-23 | Admin | View sales and product-performance reports | Correct revenue, AOV, and per-product figures shown | Pass |
| TC-24 | Responsive UI | Load the storefront on a mobile viewport | Layout reflows correctly; no horizontal overflow | Pass |

# 12. Deployment

SmartCart is deployed as four independently hosted, cloud-native services rather than a single monolithic host, reflecting how a production application is typically operated.

| Component | Provider | Notes |
|---|---|---|
| Frontend (React build) | Vercel | Automatic builds from the frontend repository; serves the production SPA |
| Backend (Spring Boot API) | Render | Hosts the REST API and Swagger UI; stateless, horizontally scalable |
| Database | Railway | Managed MySQL 8 instance used by the production backend |
| Image Storage | Cloudinary | All product images; only the secure URL is persisted in MySQL |
| Email Delivery | Gmail SMTP | Order-confirmation email notifications |

Because product images are no longer written to local disk (the project's original local-uploads approach was replaced with Cloudinary, discussed further in Chapter 14), the backend container itself is fully stateless — it can be redeployed or restarted on Render at any time without losing previously uploaded images.

The live deployment can be evaluated directly:

- Frontend: smart-cart-opal-sigma.vercel.app
- Backend API base: smartcart-backend-2muc.onrender.com
- Swagger UI: smartcart-backend-2muc.onrender.com/swagger-ui/index.html

## 13. Performance Optimizations

- Route-based code splitting and React lazy-loading, so customers loading the storefront do not pay the bundle cost of the entire admin console.
- Cloudinary transformations (f_auto, q_auto, responsive sizing) so each device downloads an appropriately sized, optimally compressed image instead of one oversized master file.
- useMemo applied around expensive client-side filtering/derivation so re-renders do not repeatedly recompute filtered product lists.
- Server-side pagination (GET /api/products/paged) to avoid transferring the entire catalogue to the client at once.
- React-Select's built-in search disabled on lower-powered/mobile devices to reduce input-driven re-renders and GPU load.
- Production environment variables used to disable verbose logging and development-only tooling in the deployed build.

# 14. Challenges & Solutions

### Stateless image storage on ephemeral hosting

The project initially stored uploaded product images on the backend's local filesystem. Because Render's filesystem is ephemeral — any files written locally are lost on redeploy or restart — this was not viable for production. The solution was to move image storage entirely to Cloudinary: the frontend uploads directly through a dedicated endpoint, Cloudinary returns a permanent secure URL, and only that URL is persisted in MySQL, making the backend container fully stateless.

### Keeping checkout atomic

Checkout touches several tables at once — reading the cart, validating and applying a coupon, creating the order and its line items, and (implicitly) affecting stock — so a partial failure partway through could leave inconsistent data. Wrapping the entire checkout method in a single @Transactional boundary ensures the order, its items, and the coupon discount are all committed together or not at all.

### Simulating a realistic payment gateway

Integrating a live payment gateway (Stripe/Razorpay) was out of scope for the current submission, but the assignment still called for payment-status tracking and failure handling. The payment service was built to model a real gateway's behaviour — recording a payment attempt, assigning a randomised outcome with an 85% success probability, and propagating SUCCESS/FAILED status back to both the payment record and the parent order — so the order-status and error-handling logic can be demonstrated honestly without a live payment integration.

### Cross-origin requests between independently hosted services

With the frontend on Vercel and the backend on Render, every API call is cross-origin. CORS was configured explicitly (rather than left permissive) to allow only the known local-development and production frontend origins, with credentials enabled, so authenticated requests succeed in production without exposing the API to arbitrary origins.

### Consistent UI across a large surface area

With over fifteen distinct customer and admin screens, keeping the glassmorphism visual language consistent required extracting shared primitives — GlassCard, LoadingOverlay/LoadingSpinner, ConfirmDialog, Pagination, and a shared PageLoader/ProtectedRoute wrapper — early on, rather than restyling each page independently.

# 15. Future Enhancements

### Payments

- Integrate a live payment gateway (Stripe and/or Razorpay) in place of the current simulated payment flow.

### Performance & Infrastructure

- Introduce Redis for caching frequently read data such as product listings and dashboard aggregates.
- Containerise the application with Docker and Docker Compose for consistent local and production environments.
- Set up GitHub Actions for CI/CD, and Kubernetes for orchestrated, horizontally scaled deployment.

### Quality & Observability

- Build out a full JUnit/Mockito unit- and integration-test suite to replace the current manual testing process.
- Add Prometheus and Grafana for application metrics and monitoring dashboards.

### Product Experience

- Convert the frontend into a installable Progressive Web App (PWA) with offline support.
- Add a user-toggleable dark/light mode alongside the current dark theme.
- Introduce AI-based product recommendations and push notifications for order and price updates.

# 16. Conclusion

SmartCart demonstrates a complete, end-to-end full stack e-commerce platform built with Spring Boot, React, and MySQL — covering secure authentication and RBAC, a normalised relational schema, a 49-endpoint documented REST API, cloud image storage, PDF invoicing, email notifications, an analytics-driven admin dashboard, and a responsive, consistently styled interface. Rather than remaining a local-only exercise, the finished system is deployed across four independent cloud providers and is reachable as a live application, satisfying both the functional modules and the cloud-deployment expectations set out in the original project brief.

Beyond meeting the assignment's evaluation criteria, the project reflects genuine production-engineering decisions — stateless image handling suited to ephemeral hosting, transactional checkout, explicit CORS and RBAC configuration, and honest handling of a simulated payment flow — and leaves a clear, prioritised path (Chapter 15) toward the remaining production-hardening work: real payment integration, automated testing, containerisation, and CI/CD.

# References


- Spring Boot Reference Documentation — https://docs.spring.io/spring-boot/
- Spring Security Reference Documentation — https://docs.spring.io/spring-security/reference/
- React Documentation — https://react.dev
- React Router Documentation — https://reactrouter.com
- MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/
- JJWT (Java JWT) Documentation — https://github.com/jwtk/jjwt
- Cloudinary Developer Documentation — https://cloudinary.com/documentation
- iText7 Documentation — https://itextpdf.com/resources/api-documentation
- springdoc-openapi Documentation — https://springdoc.org
- Vite Documentation — https://vitejs.dev
- Vercel, Render, and Railway platform documentation (frontend, backend, and database hosting respectively)

# Appendix A: Links & Repository

| Resource | Link |
|---|---|
| Source Code (GitHub) | github.com/NeoAnthem/SmartCart |
| Live Frontend | smart-cart-opal-sigma.vercel.app |
| Live Backend API | smartcart-backend-2muc.onrender.com |
| Swagger / API Docs | smartcart-backend-2muc.onrender.com/swagger-ui/index.html |
| Developer LinkedIn | linkedin.com/in/darshan-takarkhede-168937252 |